

Global low-rank LM head

Method ablations, inference bottlenecks and preliminary dataset transfer

Mean speedup versus dense: CODI 0.97-1.04x | Explicit CoT 0.99-1.15x

Head compression gives little end-to-end benefit for CODI. Explicit CoT has larger apparent gains, but much of the free-generation advantage disappears when output tokens are held fixed. The transformer remains the main runtime cost. [A, B]

1. Method and comparison setup

Both reasoning paths use the released GPT-2 CODI checkpoint. **Explicit CoT** generates reasoning as visible tokens, invoking the vocabulary head at each step. **CODI** performs six latent steps without the vocabulary head, then generates a short visible answer. Its head has fewer opportunities to save time.

The dense readout maps a 768-dimensional hidden state to about 50,259 vocabulary scores. The global rank-96 head replaces it with two learned affine maps, **768 -> 96 -> vocabulary**. One fitted head is used across all visible positions within each mode; it is not tied to an individual question.

Head fitting uses 1,024 / 256 / 256 disjoint GSM8K train questions for fitting / selection / recovery, activation-whitened initialization, four clean epochs and two recovery epochs. Nested ranks 32/64/96 are part of training; all compressed deployment arms below use rank 96. [A]

What is ablated in the first notebook?

Arm	Definition
Dense	Original full vocabulary projection followed by argmax; the latency and accuracy reference.
rank96_eager	Two low-rank linear maps and argmax in ordinary PyTorch.
rank96_compiled	The same fitted head and argmax compiled with torch.compile; the transformer itself is not compiled in this experiment.
rank96_triton_legacy	Low-rank coordinates plus a custom tiled vocabulary projection and argmax; retains the earlier lower-precision multiplication path.
rank96_triton_fp32	The same tiled selector with FP32 products/reduction, then rounding scores to the eager output dtype before selection. Weights remain FP16.

These are **deployment implementation and numerical-path ablations**, not independent loss, initialization or training-data ablations. All four compressed arms within a mode start from the same fitted head. [A]

Two protocols and three batch sizes

Free generation: each head chooses its own tokens and stopping point. **Fixed replay:** every head is evaluated, but dense-reference tokens are fed back, preserving the same trajectory and number of steps. Both protocols use batches 1, 8 and 32, 64 timing questions and five repeats on a T4 in FP16. [A]

$\text{mean_speedup_vs_dense} = \text{mean dense ms/question} / \text{mean arm ms/question}$, matched by mode, protocol and batch. Above 1 is faster; below 1 is slower. Batched ms/question is amortized batch runtime, not single-request latency.

2. Head ablations: what the speedups mean

The 60-row timing table is summarized below using only **mean_speedup_vs_dense**. Each interval spans batch sizes 1, 8 and 32; it is a measured minimum-to-maximum range, **not a confidence interval**. [A]

Rank-96 arm	CODI free	CODI replay	Explicit free	Explicit replay
Eager	0.984-1.021x	1.010-1.030x	1.079-1.145x	1.000-1.018x
Compiled	0.981-1.003x	0.997-1.033x	1.065-1.148x	1.006-1.011x
Triton legacy	0.971-0.999x	0.995-1.038x	1.083-1.132x	0.997-1.021x
Triton FP32	0.970-1.014x	0.987-1.035x	1.072-1.130x	0.988-1.014x

CODI: the overall 0.970-1.038x range is essentially flat, with small gains and regressions. **Explicit**: free generation reaches 1.065-1.148x, while fixed replay spans only 0.988-1.021x. No compressed implementation consistently produces a large trajectory-controlled improvement.

Shorter generations explain much of the explicit advantage

Explicit, batch 1	Dense ms/q	Rank-96 ms/q	Tokens: dense / rank96	Speedup
Eager, free generation	249.02	229.46	23.36 / 21.95	1.085x
Eager, fixed replay	249.84	245.43	23.36 / 23.36	1.018x

In this example, free-generation latency falls by **7.85%**, while mean visible tokens fall by **6.02%**. With dense tokens replayed, the latency reduction is only **1.76%**. Similarly, the best headline explicit result (compiled, batch 32) falls from **1.148x free** to **1.011x replay**.

This strongly supports output length and trajectory as major contributors to the apparent gain. It does not identify an exact causal percentage: prompt prefill, batch padding and longest-sequence effects also affect elapsed time. Five timing repeats do not represent five independently fitted models.

Accuracy prevents treating shorter output as a free improvement

GSM8K test, n = 1,319	Dense	Rank 96	Change (pp)
CODI: all rank-96 arms	43.52% (574)	42.99% (567)	-0.53
Explicit: eager / compiled / FP32	45.64% (602)	43.06% (568)	-2.58
Explicit: Triton legacy	45.64% (602)	43.37% (572)	-2.27

Quality uses batch 16 in notebook A. On the **full test set**, explicit mean output length is 24.40 tokens for dense and about 24.54-24.55 for rank 96. Thus, the shorter-output observation applies to the timing sample, not universally to GSM8K. These results must not be pooled with the separate batch-1 dataset run on page 5. [A]

The practical head gain is small once generation is controlled. Free-generation speed must be reported together with output length and answer accuracy.

3. Why the transformer dominates runtime

The second notebook attributes approximately **90-92% of pre-optimization diagnostic time** to the transformer. After optimization it still accounts for about **96%**. Both breakdowns below use the rank-96 head and only two diagnostic questions per mode. They are instrumented elapsed times, not pure kernel execution times. [B]

Component	Explicit before	CODI before	Explicit after	CODI after
Total (ms/question)	231.341	97.377	139.228	51.857
Transformer (ms/question)	208.339	89.408	134.110	49.728
Rank-96 head + argmax (ms/question)	4.652	0.469	1.735	0.148
Transformer fraction	90.1%	91.8%	96.3%	95.9%

Parameter counts describe storage; MAC counts describe only some work

Assume batch 1, hidden width $d = 768$, vocabulary $V = 50,259$, rank $r = 96$ and a cached attention context of $S = 100$ positions. These estimates cover one next-position decode and omit biases, normalization, GELU, softmax and cache operations.

Component	Major linear weights	Approximate MACs / call
Dense LM head	$dV = 38.60M$	$dV = 38.60M$
One transformer block	$12d^2 = 7.08M$	$12d^2 + 2Sd = 7.23M$
All 12 transformer blocks	$144d^2 = 84.93M$	$12(12d^2 + 2Sd) = 86.78M$
Rank-96 LM head	$r(d + V) = 4.90M$	$r(d + V) = 4.90M$

The seeming contradiction applies to **one block versus the dense head**: the dense head has about 5.3 times as many MACs. The complete stack already has about 2.25 times the dense head's MACs, and one block has more MACs than the compressed head. Prompt prefill and CODI's six head-free latent passes further increase the transformer's share.

Why equal MAC counts do not imply equal latency

The dense head is one large matrix-vector projection with contiguous weight access. A block chains normalization, QKV projection, attention, output projection, residual addition, normalization, FC1, GELU and FC2. At batch 1, these smaller dependent operations incur launches, CPU dispatch gaps, intermediate memory traffic and limited GPU utilization. The next operation or layer often needs the preceding result. [D]

The head is not a perfect compute-bound workload either: a single-token matrix-vector product has low arithmetic intensity and may be limited by weight reads. At an assumed 320 GB/s, reading FP16 weights alone gives optimistic traffic-only times of **241 microseconds for the dense head**, **44 for one block**, and **31 for rank 96**. These are not predictions of measured latency; they ignore other traffic and overhead. [E]

Optimize what dominates measured execution. Parameter count alone cannot identify the bottleneck, and compressed-head timings must not be compared against dense-head MAC counts.

4. Execution overhead: direct experimental evidence

Equal arithmetic, different execution structure

7,077,888 MACs per invocation	Ordinary launch (microseconds)	CUDA Graph (microseconds)
One 768 x 9,216 projection	77.13	76.95
Twelve 768 x 768 projections	129.53	110.88

Splitting the same input/weights into twelve calls costs **1.68x** as much time here; the ratio is **1.44x** with CUDA Graphs. This demonstrates a fragmentation penalty in this microbenchmark, **not a universal 10x hardware tax**. Shapes, occupancy and cache behavior also change, so the result does not isolate launch overhead alone. [B]

Isolated original transformer block: substeps

Substep (one cached position in block 1)	Mean microseconds
LayerNorm 1	55.66
QKV projection	108.67
Attention output projection	103.41
Attention/cache/reshape	266.67
LayerNorm 2	52.23
FC1	114.86
GELU	143.46
FC2	104.58
MLP overhead / unassigned time	130.87
Residuals / block dispatch	251.49
Total block (includes probe overhead)	1331.91

These are 12 instrumented repetitions. Parent times are made exclusive to avoid double counting. MLP overhead is total MLP time minus FC1, GELU and FC2; residual/dispatch and other entries include runtime and probe overhead. The **1.332 ms block total is not clean block latency**. Only the original block has a saved substep table. [B]

Optimizing the decoder produces larger gains than replacing its head

Mode / head	Clean ms/question: before -> after	Speedup
Explicit / dense	253.25 -> 146.66	1.73x
Explicit / rank 96	240.57 -> 133.18	1.81x
CODI / dense	94.18 -> 52.25	1.80x
CODI / rank 96	93.84 -> 51.90	1.81x

Selected paths: static cache + compiled steps + CUDA Graphs for explicit; the corresponding packed-INT8 path for CODI. These optimize the decode path, including head dispatch. Timing uses 16 questions x 3 repeats without hooks. Dense explicit length stays 23.88 tokens; CODI stays 2.12. The full optimized accuracy evaluation is **unfinished in the saved notebook**, so quality preservation is not established. [B]

5. GPT-2 across datasets

The planned grid is GPT-2, SmolLM2-135M and Qwen2.5-0.5B across GSM8K, SVAMP and ASDiv. Only the completed **GPT-2** results supplied in the run logs are reported here. Dense and rank-96 heads use eager FP16, batch 1; head fitting remains on GSM8K training data. [C]

Answer accuracy before and after head compression

Dataset	Mode	N	Dense	Rank 96	Delta pp
GSM8K	Explicit	1319	45.79% (604)	43.21% (570)	-2.58
GSM8K	CODI	1319	42.00% (554)	42.00% (554)	+0.00
SVAMP	Explicit	1000	41.90% (419)	38.30% (383)	-3.60
SVAMP	CODI	1000	40.20% (402)	40.10% (401)	-0.10
ASDiv*	Explicit	2084	56.77% (1183)	51.58% (1075)	-5.18
ASDiv*	CODI	2084	52.88% (1102)	52.59% (1096)	-0.29

Clean latency and generated length

Dataset	Mode	Mean ms/question: dense -> rank96	Speedup	Visible tokens
GSM8K	Explicit	217.48 -> 200.85	1.083x	23.9 -> 22.8
GSM8K	CODI	78.56 -> 78.23	1.004x	2.1 -> 2.1
SVAMP	Explicit	112.65 -> 108.17	1.041x	12.4 -> 12.4
SVAMP	CODI	78.55 -> 78.48	1.001x	2.2 -> 2.1
ASDiv*	Explicit	88.28 -> 80.09	1.102x	9.6 -> 9.0
ASDiv*	CODI	77.28 -> 76.67	1.008x	2.0 -> 2.0

CODI: accuracy changes by 0.00, -0.10 and -0.29 percentage points, with less than 1% latency reduction. **Explicit:** latency falls by 7.65%, 3.98% and 9.28%, but accuracy drops by 2.58, 3.60 and 5.18 points. GSM8K and ASDiv outputs shorten; SVAMP's displayed lengths are unchanged at one decimal. The gains are not uniformly accuracy-neutral.

* ASDiv evaluates 2,084 single-number-answer questions, not the complete 2,305-question corpus. The 221 other answer formats are excluded. Rank-96 explicit hits the token cap on 3 GSM8K and 1 ASDiv question; corresponding dense runs have none. Reported profiler inflation is 1.20-1.23x; the table above uses unprofiled timings. [C]

Conclusion and evidence limits

CODI's head-free latent work limits readout compression's impact. Explicit gains must be assessed alongside shorter outputs and accuracy loss. Decoder optimization produces the larger measured speedups; cross-model results and final optimized accuracy remain open.

Single-seed timing ranges do not establish significance. Sources use different populations, batches and timing boundaries, so their absolute times and accuracy counts are kept separate.

Sources

[A] [notebookec6a134f8f \(1\).ipynb](#): saved cell 13 timing/quality tables; cells 1 and 11 protocol and arm definitions. Run b64a76899bce85ad.

[B] [notebook1014a3299f.ipynb](#): cells 9/11 microbenchmarks, 17 clean timings, 19 breakdowns; cell 21 incomplete accuracy. Cell numbers are zero-based. [C] User-supplied GPT-2 logs: GSM8K lines 48-53, SVAMP 73-77, ASDiv 117-122.

[D] [PyTorch, Accelerating Generative AI II: GPT, Fast](#) - CPU launch overhead, matrix-vector decoding and memory traffic. [E] [NVIDIA T4 specifications](#) - 320+ GB/s memory bandwidth. External sources explain mechanisms; reported experimental numbers come from A-C.